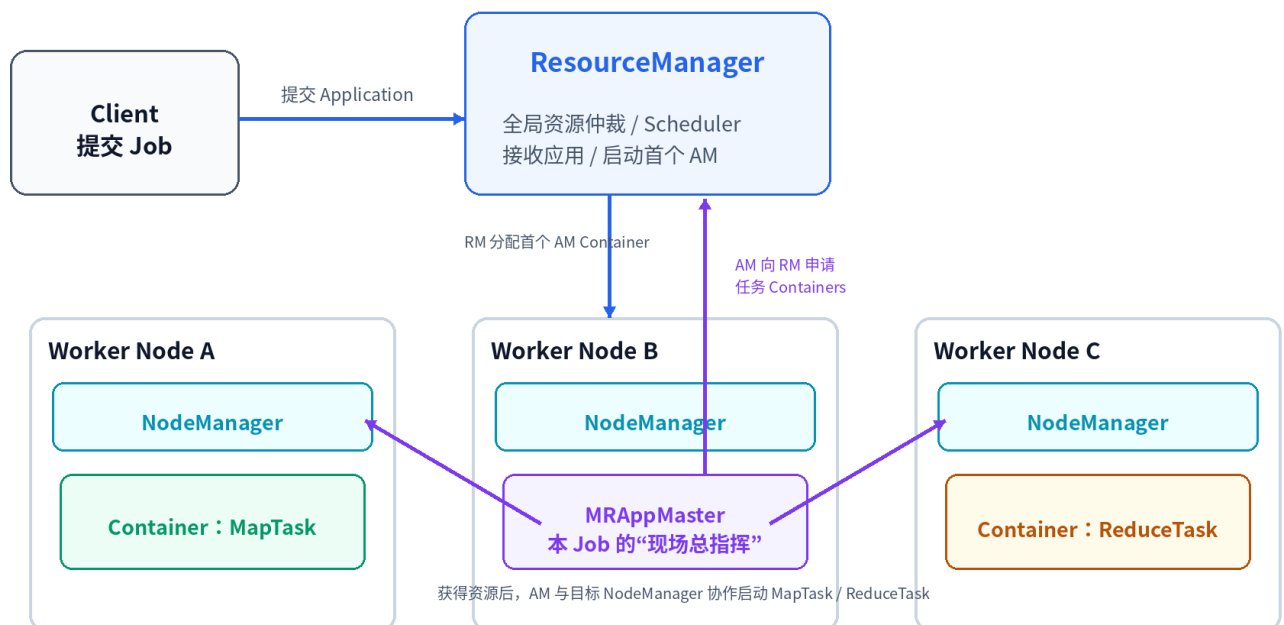


Hadoop 中的 MapReduce 原理详解

第五讲：MapReduce + YARN 的完整 Job 生命周期

Client · ResourceManager · MRAppMaster · NodeManager · Container · TaskAttempt

YARN + MapReduce：谁负责“管资源”，谁负责“干活”？



目标：看懂你执行一条 `hadoop jar` 命令后，集群内部到底发生了什么

学习目标

前四讲已经解释了数据如何进入 Mapper、Map 端如何产生中间结果，以及 Shuffle/Reduce 如何重组数据。本讲换一个视角：不再只看“数据怎么流”，而是看“谁来组织这些计算”。这就是 YARN 的控制面。

- 分清 HDFS、YARN、MapReduce 三者的职责：存储、资源管理、计算模型。
- 掌握 Client、ResourceManager、NodeManager、MRAppMaster、Container 五个核心角色。
- 从 `hadoop jar / Job.waitForCompletion()` 开始，完整追踪一个 Job 的提交、调度、执行与结束。
- 理解 Container 是“资源配额 + 执行位置”，不是一台机器，也不等同于 Docker。
- 理解为什么 MapTask 调度强调数据本地性，而 ReduceTask 天生需要跨节点 Shuffle。
- 区分 Job、Application、Task、TaskAttempt、Container 这些最容易混淆的名词。

- 理解任务失败、节点故障后为什么可以重新执行，以及 Map 中间输出丢失时会发生什么。

先建立总框架

前四讲主要是“数据平面”：数据怎样被切、排序、Shuffle、聚合。本讲主要是“控制平面”：谁申请资源、谁启动进程、谁监控失败、谁决定重试。把这两个平面叠在一起，MapReduce 才是完整的分布式系统。

0. 先分清三套东西：HDFS、YARN、MapReduce

初学 Hadoop 最容易把三个名字混成一个。实际上它们解决的是三个不同问题。

组件	它主要回答的问题	典型对象
HDFS	海量数据放在哪里？怎样可靠保存？	NameNode、DataNode、Block、Replica
YARN	集群 CPU/内存应该分给谁？任务在哪台机器启动？	ResourceManager、NodeManager、ApplicationMaster、Container
MapReduce	拿到数据和计算资源后，怎样按 Map → Shuffle → Reduce 处理？	Mapper、Partitioner、Reducer、InputFormat、OutputFormat

一句话记忆

HDFS = 仓库；YARN = 集群资源调度中心；MapReduce = 一种批处理计算方法。MapReduce 通常“跑在 YARN 上”，输入和输出通常“放在 HDFS 里”。

1. YARN 为什么出现？它到底解决什么问题？

Hadoop 1.x 时代有一个很重的 JobTracker：既要管理整个集群的资源，又要负责每个 MapReduce Job 的任务调度、状态跟踪和容错。集群越来越大以后，这个角色容易成为瓶颈。YARN 的核心改造是“职责拆分”。

职责	Hadoop 1.x	YARN / MRv2
全局资源管理	JobTracker	ResourceManager
每台机器上的执行代理	TaskTracker	NodeManager
每个 Job 自己的协调者	主要压在 JobTracker 上	每个应用自己的 ApplicationMaster
任务运行资源	固定 map/reduce slot 思路	Container：更通用的资源抽象

因此 YARN 的本质不是“另一个 MapReduce 算法”，而是把集群改造成一个通用资源平台。MapReduce、Spark 等框架都可以在 YARN 上申请资源运行。

YARN + MapReduce：谁负责“管资源”，谁负责“干活”？

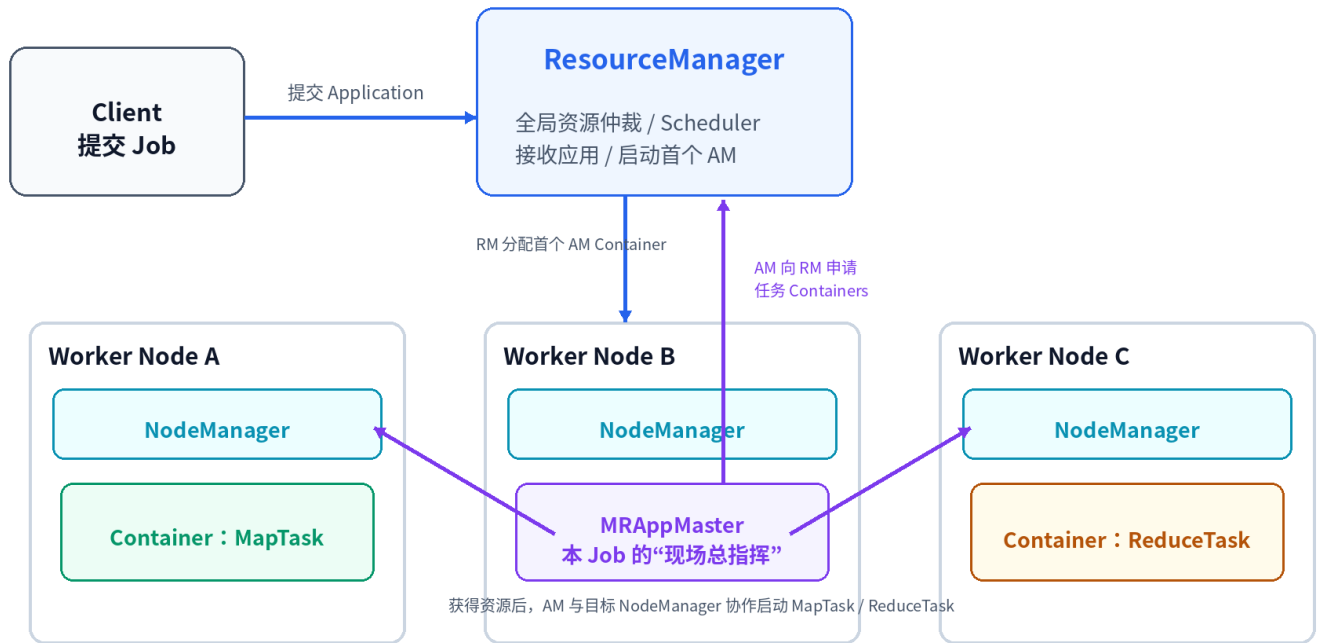


图 1：YARN 的核心角色。ResourceManager 管全局资源；MRAppMaster 只负责自己的这个 Job；NodeManager 管本机。

2. 五个核心角色逐个讲清

2.1 Client：提交者，不是“真正干活的计算节点”

你在终端执行 `hadoop jar ...`，或者 Java 程序调用 `job.waitForCompletion(true)`，首先工作的就是客户端代码。Client 负责把“我要算什么、输入在哪、输出在哪、用哪个 Mapper/Reducer、jar 在哪”整理成一个 Job。

官方 Hadoop 3.3.5 教程把提交阶段概括为：检查输入/输出、计算 `InputSplit`、准备 `DistributedCache` 信息、把 jar 和配置复制到文件系统上的 MapReduce 系统目录，然后向 ResourceManager 提交 Job。

2.2 ResourceManager：全局资源仲裁者

ResourceManager (RM) 站在整个 YARN 集群的全局视角。它知道有哪些 NodeManager、哪些节点还有 CPU/内存、哪些应用正在排队。它的 Scheduler 按队列、优先级和资源请求分配 Container。

一个容易犯的错

ResourceManager 不会亲自逐条运行 `map()` 或 `reduce()`。它主要解决“资源给谁、给多少、放在哪个节点”这个问题。

2.3 NodeManager：每台 Worker 机器上的“现场管理员”

每台参与 YARN 计算的 Worker 节点通常运行一个 NodeManager (NM)。它负责接收启动/停止 Container 的请求、准备本地资源、启动任务进程、监控 CPU/内存等资源使用，并周期性向 ResourceManager 汇报节点状态。

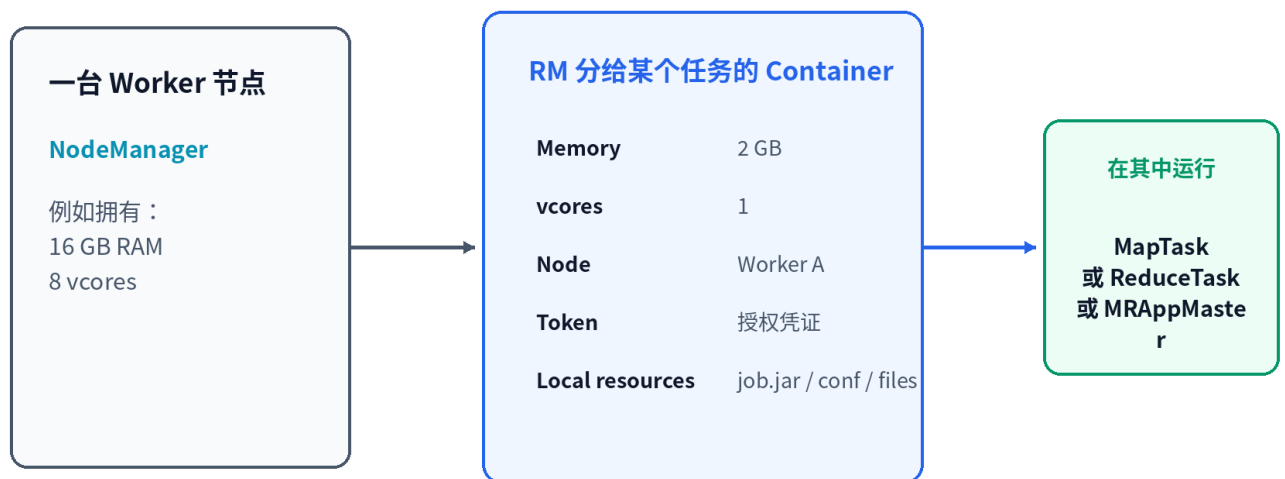
2.4 MRAppMaster：每一个 MapReduce Job 自己的总指挥

对于 MapReduce，ApplicationMaster 的具体实现通常叫 MRAppMaster。它不是整个集群只有一个，而是“每个正在运行的 MapReduce Application 通常有自己的一个”。它知道这个 Job 有多少 Map/Reduce task、哪些完成了、哪些失败了、还需要向 RM 申请多少 Container。

2.5 Container：资源配额 + 执行位置

Container 可以理解成 YARN 发出的一张“资源使用券”：某台节点上给你一定内存、一定 vcores，并附带身份/授权信息。随后 ApplicationMaster 与目标 NodeManager 交互，让 NodeManager 在这份资源额度里启动 MapTask、ReduceTask 或其他进程。

YARN Container：不是“另一台机器”，而是一份资源配额 + 执行位置



关键：YARN Container 是资源抽象。默认并不等于 Docker 容器；它规定“给这个进程多少资源、在哪个节点运行”。

图 2：YARN Container 是资源抽象。默认意义上不要把它直接等同于 Docker 容器。

3. 你敲下 `hadoop jar` 以后：提交前到底做了什么？

终端里看起来只有一条命令：

```
# 典型提交命令（示意）
hadoop jar myjob.jar com.example.WordCount /input /output
```

但客户端在真正把 Job 交给 ResourceManager 之前，要先把很多“准备工作”做完。最重要的有下面几步。

3.1 校验输入与输出

例如输入路径是否存在、OutputFormat 是否接受当前输出路径。经典 FileOutputFormat 通常要求最终输出目录事先不存在；否则 Job 在正式跑之前就可能报错。这样可以减少长时间计算后才发现输出路径冲突的风险。

3.2 计算 InputSplit

第二讲讲过：InputSplit 是“一个 Mapper 的逻辑输入范围”。官方 3.3.5 教程明确把“计算 InputSplit”列在 Job submission 阶段。这意味着在常见文件输入场景中，Job 还没有正式执行 MapTask，客户端就已经形成了任务切分信息。

3.3 把 Job 运行所需材料放到分布式可访问的位置

集群中的 Worker 节点必须能拿到你的 jar、配置和必要资源。因此客户端会把这些材料复制到文件系统中的提交/系统目录。随后具体节点在任务启动时再把所需资源本地化。

为什么不能只放在你的电脑 C:\Users\... 里？

因为真正运行 MapTask 的可能是远程 Worker。远程机器看不到你本机桌面上的 jar。分布式系统首先要解决的就是“代码和配置怎样送到计算节点”。

4. 完整 Job 生命周期：12 步把控制流走一遍

一次 hadoop jar 背后的完整生命周期（控制流）



图 3：一次 MapReduce Job 从 Client 到 ResourceManager、MRAppMaster、NodeManager，再到最终输出的控制流程。

第 1-3 步：Client 准备并提交 Application

Client 构造 Job，验证输入输出，计算 Split，上传 jar/配置等，然后把应用提交给 ResourceManager。此时可以把它理解成“把一张完整的工程订单交给调度中心”。

第 4 步：先给 MRAppMaster 找一个 Container

ResourceManager 的 ApplicationsManager 接受应用以后，需要先让这个应用有一个自己的协调者。RM 为 ApplicationMaster 协商第一个 Container，并让相应 NodeManager 启动 MRAppMaster。

第 5 步：MRAppMaster 初始化这个 Job

MRAppMaster 启动后会注册到 RM，读取 Job 的配置与任务信息，建立任务状态。例如：有 8 个 Map task、2 个 Reduce task；Map 0 还没跑，Map 1 已分配等。

第 6-7 步：MRAppMaster 向 RM 申请任务 Container

MRAppMaster 不直接“抢机器”。它向 RM 提出资源请求：我还需要若干 Container，每个大约多少内存、多少 vcores；对于 Map，还会附带数据位置偏好。Scheduler 根据全局资源状况进行分配。

第 8 步：MRAppMaster 让 NodeManager 启动任务

RM 返回“你获得了哪些 Container”以后，MRAppMaster 再与这些 Container 所在节点的 NodeManager 交互，准备启动上下文、环境变量、本地资源和命令，随后真正启动 TaskAttempt。

第 9-10 步：真正的数据计算

MapTask 开始读自己的 InputSplit，经过 Mapper、Spill、Partition、Sort 形成本地中间结果。ReduceTask 启动后通过 Shuffle 服务 Fetch 各 Mapper 中属于自己的 partition，再 Merge/Sort/Group，最后调用 reduce()。这部分正是前四讲的“数据平面”。

第 11-12 步：提交输出并结束应用

成功的任务通过 OutputCommitter/RecordWriter 等机制形成最终输出。Job 完成后 MRAppMaster 汇报最终状态并结束，YARN 释放相关资源；Client 的 waitForCompletion(true) 才真正返回成功或失败。

5. Container 再深入一层：它为什么是 YARN 的关键抽象？

假设 Worker A 有 16 GB 内存和 8 个 vcores。YARN 不希望一个 Job 把整台机器霸占，也不希望所有任务无序地抢资源。因此 Scheduler 把节点资源切成可分配的额度。Container 就是这个额度的逻辑载体。

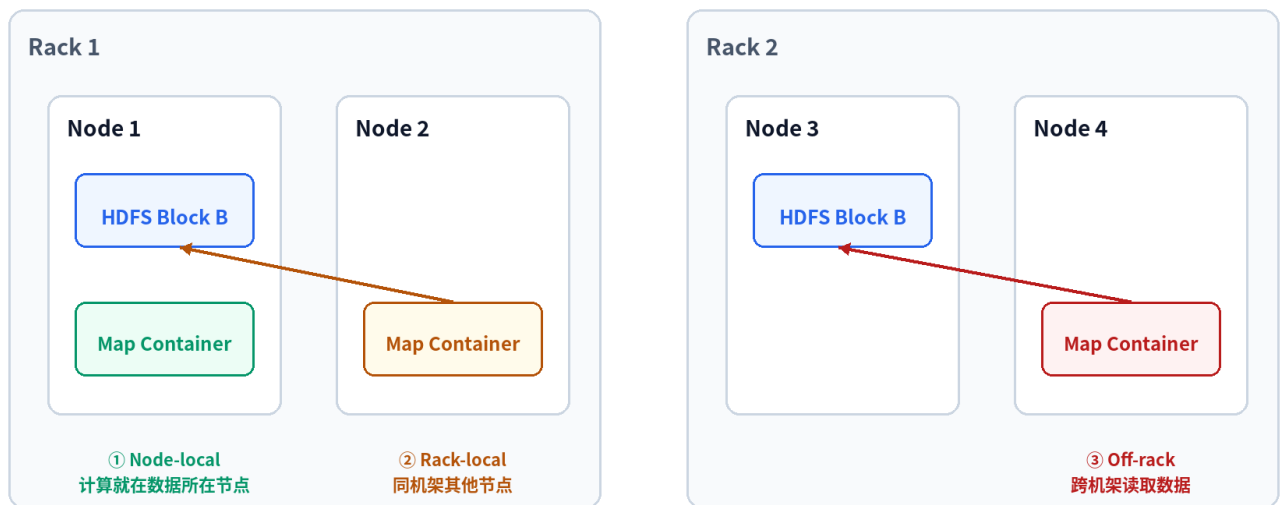
你可能的错误理解	更准确的理解
一个 Container = 一台机器	一个节点上可以同时存在多个 Container，只要资源允许
Container = Docker	YARN Container 首先是资源分配/执行抽象；可以配置不同运行时，但概念并不等价
MapTask 固定只能用 1 GB	资源由配置与调度策略决定，可为 map/reduce 分别配置内存/vcores
RM 直接启动所有进程	RM 负责分配；ApplicationMaster 与 NodeManager 协作启动具体任务

所以 YARN 把“框架逻辑”和“资源”解耦了。MapReduce 只需要告诉 YARN：我需要这些计算资源；至于整个集群怎样公平分配，由 YARN 负责。

6. 为什么 MapTask 调度时要尽量靠近数据？

HDFS 和 MapReduce 经常部署在同一批机器上，目的是让“存数据的节点也能计算”。一个 Split 通常对应某些 HDFS Block 的位置。MRAppMaster 申请 Map Container 时，可以把这些位置作为 locality 偏好告诉 Scheduler。

Map 任务为什么要讲“数据本地性”：移动计算，比搬 128 MB 数据更便宜



调度目标不是“永远本地”，而是尽量本地：等待太久也会浪费集群资源，因此调度器会在 locality 与及时启动之间权衡。

图 4：Node-local 最理想，其次 Rack-local；跨机架读取通常意味着更高网络成本。

6.1 Node-local：计算就在数据所在节点

如果 Node 1 本地已有 HDFS Block B，同时还有空闲 Container，就把对应 MapTask 放到 Node 1。此时大量数据不必穿过网络。

6.2 Rack-local：退而求其次

如果数据节点暂时没有资源，但同机架其他节点有空闲资源，可以在同机架运行并通过机架内部网络读取。通常仍比跨机架搬数据便宜。

6.3 Off-rack：为了不无限等待，必要时远程读

调度不是“为了本地性可以永远等”。如果为了等本地节点空闲而让整个集群长时间闲置，反而不划算。因此真实调度会在 locality 与任务及时启动之间折中。

为什么 Reduce 不强调同一种数据本地性？

Reducer 的输入本来就来自许多 Mapper 所在节点。它天然要做 Shuffle，所以很难像 Map 那样“让计算直接贴着一块 HDFS 输入数据”。

7. Job、Application、Task、TaskAttempt、Container：一次分清

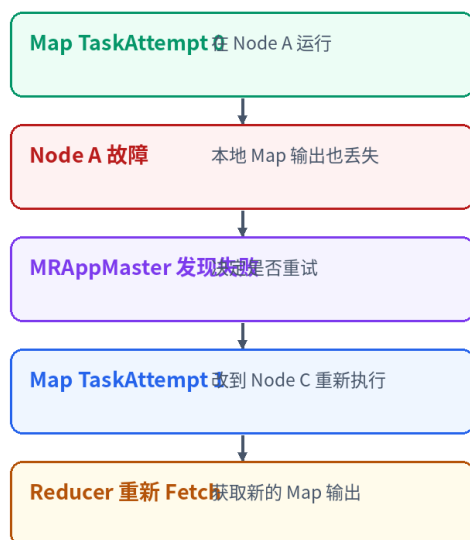
术语	你可以怎样理解	例子
Job	MapReduce 业务任务本身	“统计 1 TB 日志中每个 URL 的访问量”

术语	你可以怎样理解	例子
YARN Application	YARN 看待的一次应用运行	这个 MapReduce Job 在 YARN 中对应一次 Application
Task	逻辑任务	MapTask-0、MapTask-1、ReduceTask-0
TaskAttempt	Task 的一次具体尝试	MapTask-0 第一次在 A 节点失败，第二次在 C 节点成功
Container	某次运行被分配到的资源额度/位置	Node C 上 2 GB + 1 vcore 的执行资源

最重要的是 Task 与 TaskAttempt。Task 是“应该完成的工作”；Attempt 是“实际尝试去完成它的一次运行”。容错正是建立在“一个 Task 可以有多个 Attempt”之上。

8. 失败时系统怎样恢复？为什么可以重新算？

失败不是异常情况：MapReduce/YARN 的设计就是“失败后重新调度”



底层原则：保存原始输入（HDFS）+ 让计算可重做，比给每个临时中间结果都做昂贵的持久化复制更划算。

为什么 Map 已经“成功”还可能重跑？

因为 Map 中间输出通常在节点本地磁盘，不是 HDFS 持久副本。节点挂掉以后，Reducer 无法再 Fetch 这份中间结果。

因此系统可以：

- 重新执行丢失输出的 Map task
- 为失败的 task 启动新的 attempt
- 失败超过阈值后把整个 Job 判为失败

图 5：失败后重新调度是 MapReduce 的常态设计，而不是临时补丁。

8.1 TaskAttempt 失败

如果某个 Map/Reduce 的 JVM 异常退出、超时或被判定失败，MRAppMaster 可以为同一个逻辑 Task 发起新的 Attempt，通常换一个 Container 重新运行。失败次数超过阈值后，Job 才会被判失败。

8.2 Worker 节点挂掉

NodeManager 长时间不向 RM 汇报，节点可能被视为不可用。运行在该节点上的任务需要重新调度。更关键的是：已经“完成”的 Map 输出通常只存本地磁盘；如果节点彻底失联，这些中间结果也不可再 Fetch。

8.3 为什么 Map 可能要重跑？

因为原始输入通常在 HDFS，有副本，可以重新读取；Map 计算又被设计成可重执行。因此系统更愿意“重算中间结果”，而不是把每份 Map 临时输出都做成 HDFS 三副本。这样在正常情况下成本更低。

8.4 ApplicationMaster 自己挂了怎么办？

YARN 的 ResourceManager/ApplicationsManager 负责应用级管理，并能够为 ApplicationMaster 处理重新启动。具体重试次数与配置有关。理解重点是：ApplicationMaster 也不是被假设成永远不会故障。

9. 用 WordCount 把“控制流 + 数据流”叠在一起

假设 HDFS 中有 512 MB 文本，形成 4 个 InputSplit；Job 配置 1 个 Reducer。你执行 WordCount 后，可以把全过程想成下面这样。

时间	控制面发生什么	数据面发生什么
T0	Client 计算 4 个 Split，提交 Job	尚未真正处理文本
T1	RM 启动 MRAppMaster	MRAppMaster 获取任务计划：4 Map + 1 Reduce
T2	AM 请求 Map Containers	4 个 MapTask 尽量靠近各自 HDFS 数据启动
T3	AM 监控 Map attempts	每个 Mapper 输出 <word,1>，本地 spill/sort/partition
T4	AM 启动/管理 Reduce attempt	Reducer 从 4 个 Mapper Fetch 自己的 partition
T5	任务完成并 commit	reduce(word,[1,1,...]) 求和并写 part-r-00000
T6	AM 向 YARN 结束应用	Client 看到 Job succeeded

把前五讲连起来

InputSplit 决定“逻辑上有多少 Map 工作”；YARN Container 决定“这些工作在哪里、用多少资源运行”；Mapper/Shuffle/Reducer 决定“数据在任务内部怎样计算”。三个层次不要混在一起。

10. 在你的 Hadoop 3.3.5 虚拟机里怎样观察这些角色？

下面这组命令的目的不是“背命令”，而是把抽象角色映射到真实进程和真实 Application。每次重开虚拟机后，先启动服务并做健康检查。

A. 启动并确认核心进程

```
# 1) 启动 HDFS (如果本次虚拟机刚启动)
start-dfs.sh

# 2) 启动 YARN
start-yarn.sh
```

```
# 3) 查看 Java 进程
jps
```

常见情况下你会看到 NameNode/DataNode (HDFS) , 以及 ResourceManager/NodeManager (YARN) 。伪分布式单机环境里, 它们可能都在同一台虚拟机上, 但逻辑职责仍然不同。

B. 直接从 YARN 观察调度对象

```
# 查看 YARN 已注册的节点
yarn node -list

# 查看当前正在运行的 Application
yarn application -list

# 查看包括已结束任务在内的 Application
yarn application -list -appStates ALL
```

运行一个 Job 时, 终端通常会打印 application_... 和 job_... 之类的 ID。Application ID 是 YARN 视角; Job ID 是 MapReduce 视角。二者相关, 但不要把名称当成完全同一个概念。

C. 跑任务时观察 Application 的出现与结束

```
# 示例: 运行 Hadoop 3.3.5 自带 WordCount
# 注意: /mrdemo/output 如果已经存在, 要先删除或换一个新的输出目录。
hadoop jar \
  $HADOOP_HOME/share/hadoop/mapreduce/hadoop-mapreduce-examples-3.3.5.jar \
  wordcount /mrdemo/input /mrdemo/output

# 查看最终结果
hdfs dfs -cat /mrdemo/output/part-r-00000
```

如果你同时开另一个终端反复执行 yarn application -list, 就能看到 Application 从运行到结束。这个小实验非常适合课堂上让学生把 “MRAppMaster/Container/Task” 与真实系统对应起来。

11. 单机伪分布式环境会不会 “看不出分布式” ?

会有这种视觉错觉。你的虚拟机可能只有一个操作系统实例, 所以 ResourceManager、NodeManager、NameNode、DataNode 甚至 Map/Reduce task 都在同一台机器上。网络搬运成本也不像真正多机集群那样明显。

但教学价值仍然存在, 因为控制逻辑没有消失: RM 仍然负责资源管理, NM 仍然负责 Container, MRAppMaster 仍然协调自己的 Job; 只是这些角色被 “压缩” 到一台机器。

类比

像在一栋楼里模拟 “总部、仓库、车间、调度中心” 。虽然地理上都在一栋楼, 组织结构仍然可以完整运行; 真正多机集群只是把这些角色放到不同机器并引入真实网络与故障。

12. 常见疑问与容易理解错的地方

Q1: ResourceManager 是不是 MapReduce 的一部分？

更准确地说，它是 YARN 的核心服务；MapReduce 作为一个计算框架使用 YARN。

Q2: 一个集群只有一个 ApplicationMaster 吗？

不是。ApplicationMaster 是“每个应用自己的协调者”；同时运行多个应用时会有多个 AM。

Q3: MRAppMaster 自己也需要 Container 吗？

需要。RM 首先为 ApplicationMaster 分配一个 Container，再由它去申请后续任务 Container。

Q4: Container 是 JVM 吗？

不是同一概念。Container 是资源/执行环境抽象；里面通常会启动任务进程/JVM。

Q5: 一个 Worker 只能跑一个 Container 吗？

不是。只要资源足够，一个 NodeManager 可以并行管理多个 Container。

Q6: InputSplit 是 MRAppMaster 运行以后才计算吗？

官方 3.3.5 教程把计算 InputSplit 列在 Job submission 过程中，常见文件输入会在正式提交前形成切分信息。

Q7: Reducer 的 Container 也应该靠近 HDFS 输入吗？

没有 Map 那么直接。Reducer 输入主要是许多 Mapper 的中间结果，天然要跨节点 Shuffle。

Q8: Map 成功后为什么节点一挂还要重跑？

因为 Map 中间输出通常只在本地磁盘；节点不可达后 Reducer 无法再 Fetch。

Q9: Client 退出后 Job 一定立即停止吗？

不一定。submit() 可以提交后返回；waitForCompletion() 是客户端选择等待并监控。实际应用生命周期由 YARN 管理。

Q10: YARN 只服务 MapReduce 吗？

不是。它是通用资源管理平台，MapReduce 只是运行在 YARN 上的一类应用。

Q11: MapTask 是一个线程吗？

从框架视角它是一个逻辑 Task；具体 TaskAttempt 通常在独立任务 JVM/进程环境中执行，不应简单理解成“一个 Java 线程”。

Q12: 一个 Task 失败是不是整个 Job 立即失败？

通常不是。框架允许 TaskAttempt 重试；超过重试阈值或发生不可恢复错误后，Job 才可能失败。

Q13: ResourceManager 会不会读取每一条业务数据?

不会。RM 管资源与应用调度，不承担你的 Mapper 业务数据计算。

Q14: 为什么要同时学控制流和数据流?

因为性能和故障问题经常跨两层：数据倾斜是数据分布问题，但最终表现为某个 Reduce Container 长时间占用资源。

13. 最终心智模型：把五讲合成一张“脑内地图”

到这里你应该能同时回答两类问题。第一类是“数据在哪里、怎样变化”：HDFS → InputSplit → Mapper → Spill/Partition/Sort → Shuffle → Group → Reducer → HDFS。第二类是“这些阶段由谁组织运行”：Client → RM → MRAppMaster → NodeManager/Container → TaskAttempt。

数据平面：

HDFS → InputSplit → Map → Spill/Sort/Partition → Shuffle → Reduce → HDFS

控制平面：

Client → ResourceManager → MRAppMaster → NodeManager → Container → TaskAttempt

一句话总结

MapReduce 定义“怎么算”；YARN 决定“给谁资源、在哪里算”；HDFS 提供“数据从哪里读、结果往哪里存”。

下一讲建议继续学习 MapReduce 的“可靠性与性能”部分：Task 重试、推测执行（Speculative Execution）、数据倾斜、慢节点、Combiner、压缩，以及为什么一个 Job 经常“99% 卡很久”。这些内容会把原理直接连接到真实运维与性能调优。

资料依据

1. Apache Hadoop 3.3.5 MapReduce Tutorial（官方）：Job submission、InputSplit、任务执行与 OutputCommitter 等。
2. Apache Hadoop YARN Architecture（官方）：ResourceManager、NodeManager、ApplicationMaster、Container 的职责划分。

说明：本文以 Hadoop 3.x / YARN（MRv2）的常见工作方式为主。具体调度策略、重试阈值、资源大小与安全机制可随集群配置而变化。